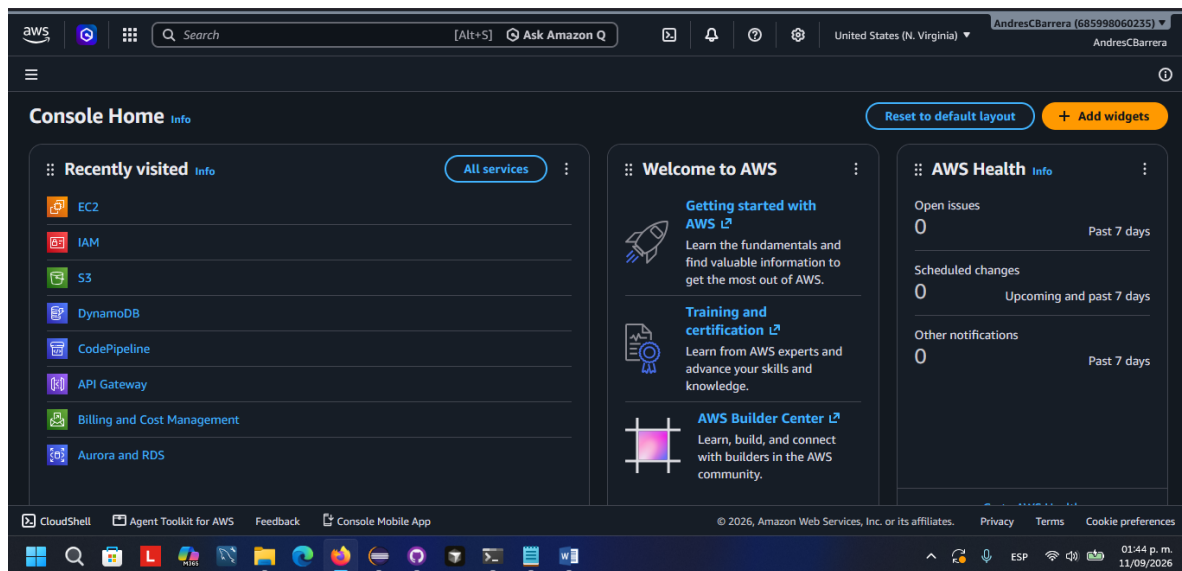


Semana 5

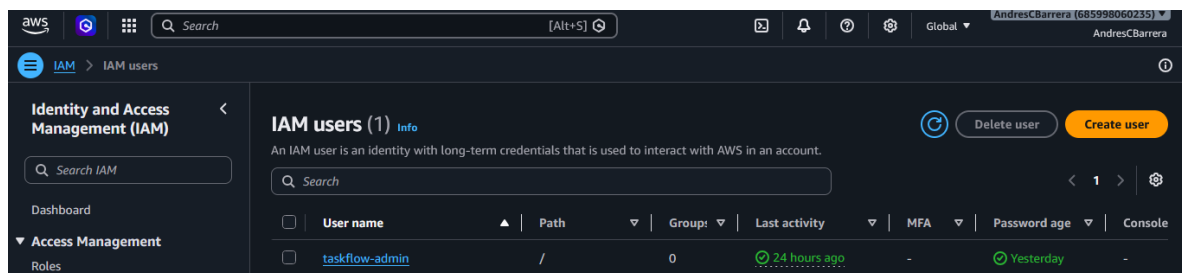
AWS y Cloud



Pantalla de Evidencia de Cuenta AWS Inicio.

A la mitad de esta semana, nos pasamos a aprender sobre los conceptos de Cloud a través de AWS (Amazon Web Services), específicamente sus servicios básicos, que son lo EC2, S3, y VPC, junto con lo que es RDS y DynamoDB para terminar con CodePipeline.

Aunque usamos la cuenta gratuita, por prevención nuestro primer paso en nuestra primera sesión fue la creación de un Budget para que no tuviéramos luego problemas en cuanto costos no planificados al mandarnos alertas automáticas sobre estos, dándonos un colchón de seguridad. De aquí, ya pasamos a la creación de un usuario IAM por estándar de seguridad. Así, aprendemos sobre el habito de manejar operaciones a través de un usuario con ciertos permisos y no solamente como un root user.



Pantalla de IAM, donde tenemos nuestros usuario para manejar cloud..

Aquí pasamos a uno de los conceptos mas importantes en cloud computing, en donde adquirimos un servidor virtual con la que cual trabajar, un Elastic Compute

AWS | | | | Search [Alt+S] | | | United States (N. Virginia) ▾ | AndresCBarrera (685998060235) ▾ | AndresCBarrera

EC2 > Instances

Instances (1/1) Info

< 1 >

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IP
☒ taskflow-ec2	i-069f00c106486249b	Running	t3.micro	3/3 checks pass	us-east-1b	ec2-44-

i-069f00c106486249b (taskflow-ec2)

< 1 >

Name	Security group rule ID	Port range	Protocol	Source
-	sgr-00ae78fb873fab22	22	TCP	189.219.66.183/32
-	sgr-095d81441e4a65600	8080	TCP	0.0.0.0/0

Se siguió con la elaboración de llaves cifrados y archivos tipo .pem para un acceso a través de consola por medio de ssh, que junto con los permisos ya definidos en la instancia en sí, aseguran nuestra instancia de trabajo.

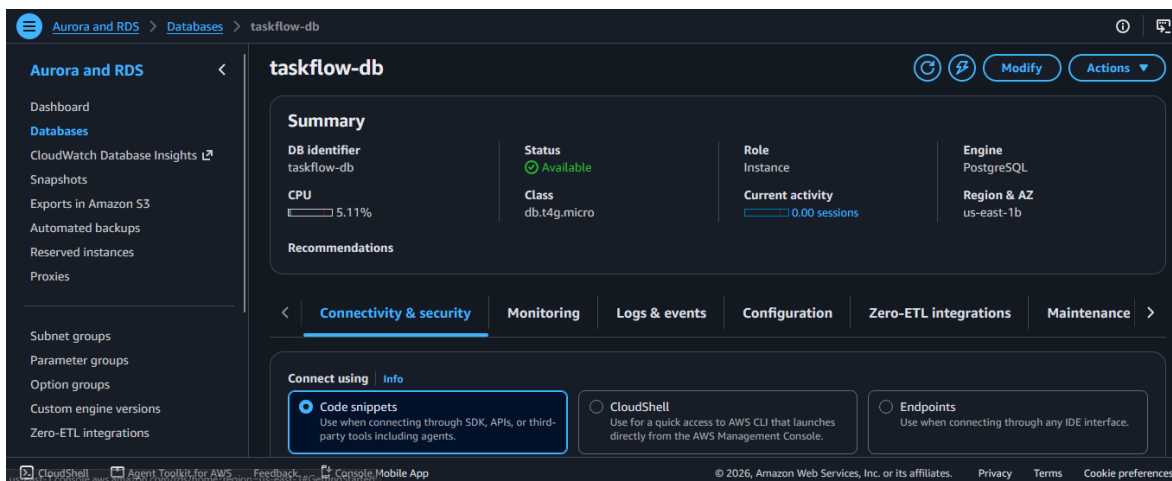
```
user@DESKTOP-9GN0P2R:~/ConexionacademyMty/EC2dia2taskflowpem$ ssh -i taskflow-key.pem ec2-user@44.206.241.91
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html

      #_
     ~\ ##### Amazon Linux 2023
    ~~~ \#####\
        \####|
        \#/ ___ https://aws.amazon.com/linux/amazon-linux-2023
         V~'  '->
            /
           /
          /
         /
        /
       /
      /
     /
    /
   /
  /
 /
/_m/'

Last login: Fri Sep 11 15:19:55 2026 from 189.219.66.183
[ec2-user@ip-172-31-82-29 ~]$ java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-82-29 ~]$ |
```

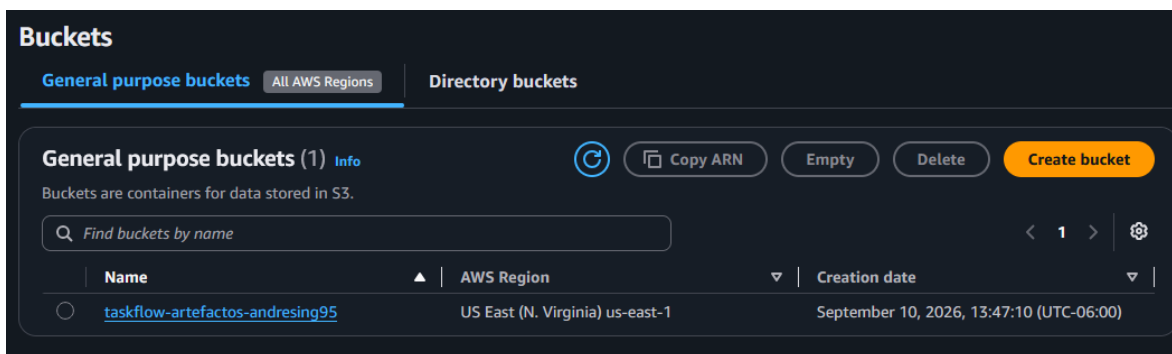
Con unos comandos, pudimos conectar directamente con esta instancia y mandar nuestro jar directamente, cosa que levanto la aplicación en <http://44.206.241.91:8080/swagger-ui/index.html> de manera pública, y ya funcionando al tener una base de datos interna pequeña h2 en base java. De aquí, pasamos a aplicar el concepto de crear una base de datos en la nube en sí, con la creación de un RDS, o una base de datos relacional con AWS. De esto también aplicamos conocimiento de VPC, o Virtual Private Cloud, que es la red privada que tiene un

usuario de AWS tiene para trabajar en la nube y donde viven nuestros recursos, demarcando así los límites entre los recursos de los demás y los nuestros.



Una base de datos relacional en la nube

Ya con esa base de datos creado, pasamos a la creación de lo que es un bucket en el Simple Storage Service o S3 de Amazon, que toma el rol de como un disco duro en la nube en donde subimos nuestro jar ya compilado de taskflow. Ajustando unos roles de seguridad con la creación de un presigned url, podemos compartir archivos guardados en la nube. Así vemos como podemos usar AWS para crear lo que es un data lake de manera virtual, en donde podemos almacenar una gran cantidad de datos de manera centralizada, con escalas que llegan hasta 50TBs.



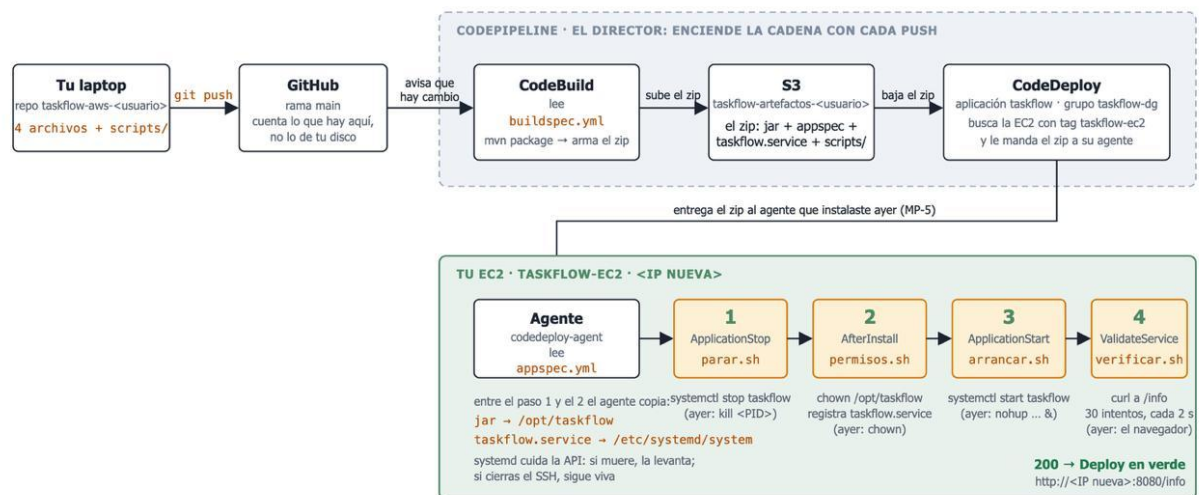
Un bucket en la nube, donde tenemos guardado la jar de nuestro taskflow para compartir.

Ya viendo esto, pasamos a conectar nuestra aplicación viviendo en EC2, a la base de datos RDS que creamos para demostrar como podemos crear arquitecturas completamente en la nube. Manejando unos comandos de seguridad en la instancia de EC2 para crear un token JWT y aparte tener unos ajustes con las reglas de seguridad para permitir que la instancia de ec2 hable con nuestro rds, corremos lo siguiente:

```
nohup java -jar taskflow-api.jar \
--spring.profiles.active=docker \
--DB_HOST=taskflow-db.cgdkeoi8tor.us-east-1.rds.amazonaws.com --DB_PORT=5432 --DB_NAME=taskflow \
--DB_USER=taskflow --DB_PASSWORD=' ' \
--JWT_SECRET=' ' \
> app.log 2>&1 &
```

Esto vuelve a levantar la aplicación ya hablando con una con la base de datos en la nube.

Luego empezamos a trabajar con los conceptos de CodeBuild, CodeDeploy, y Codepipeline.



Empezamos a ver lo que es el uso de bases de datos DynamoDB, que es perteneciente al ambiente de AWS exclusivamente como una base de datos NoSQL de clave y valor. Esto, junto con instalar y configurar la CLI (Command Line Interface) de AWS, podemos crear una tabla directamente con el siguiente comando:

```
aws dynamodb create-table --table-name taskflow-eventos --attribute-
definitions AttributeName=taskId,AttributeType=S
AttributeName=fechaHora,AttributeType=S --key-schema
AttributeName=taskId,KeyType=HASH AttributeName=fechaHora,KeyType=RANGE -
-billing-mode PAY_PER_REQUEST
```

y luego, manejando esta CLI podemos crear unas tablas directamente con en vivo.

```

user@DESKTOP-9GN0P2R:~$ aws sts get-caller-identity
{
  "UserId": "AIDAZ7OFLWLF3MSPLK3FZ",
  "Account": "685998060235",
  "Arn": "arn:aws:iam::685998060235:user/taskflow-admin"
}
user@DESKTOP-9GN0P2R:~$ aws dynamodb describe-table --table-name taskflow-eventos --query "Table.TableStatus"
"ACTIVE"
user@DESKTOP-9GN0P2R:~$ aws dynamodb put-item --table-name taskflow-eventos --item '{
  "taskId": { "S": "T-001"},
  "fechaHora": { "S": "2026-09-08T09:15:00Z"},
  "tipo": { "S": "CREADA"},
  "autor": { "S": "ana" }
}'
user@DESKTOP-9GN0P2R:~$ aws dynamodb get-item --table-name taskflow-eventos \
--key '{"taskId":{"S":"T-001"},"fechaHora":{"S":"2026-09-08T09:15:00Z"}}'
{
  "Item": {
    "autor": {
      "S": "ana"
    },
    "tipo": {
      "S": "CREADA"
    },
    "taskId": {
      "S": "T-001"
    },
    "fechaHora": {
      "S": "2026-09-08T09:15:00Z"
    }
  }
}

```

```

user@DESKTOP-9GN0P2R:~$ aws dynamodb query --table-name taskflow-eventos \
--key-condition-expression "taskId = :t" \
--expression-attribute-values '{"t":{"S":"T-001"}}' \
--return-consumed-capacity TOTAL
{
  "Items": [
    {
      "autor": {
        "S": "ana"
      },
      "tipo": {
        "S": "CREADA"
      },
      "taskId": {
        "S": "T-001"
      },
      "fechaHora": {
        "S": "2026-09-08T09:15:00Z"
      }
    }
  ],
  "Count": 1,
  "ScannedCount": 1,
  "ConsumedCapacity": {
    "TableName": "taskflow-eventos",
    "CapacityUnits": 0.5
  }
}
user@DESKTOP-9GN0P2R:~$ |

```

Los resultados de crear una DynamoDB .

Ya con esto, pasamos a la integración de lo que hemos aprendido. Creando un nuevo bucket con unos cuantos ajustes adicionales, junto con una creación de un perfil de instancia junto con más roles de seguridad para el CodePipeline para que nuestro EC2 con su agente instalado:

```

[ec2-user@ip-172-31-82-29 ~]$ sudo dnf install -y ruby wget
Last metadata expiration check: 4:11:42 ago on Thu Sep 10 18:03:05 2026.
Package wget-1.21.3-1.amzn2023.0.5.x86_64 is already installed.
Dependencies resolved.
=====
Package                                Architecture    Version                                Repository      Size
=====
Installing:
ruby3.2                                x86_64          3.2.0-184.amzn2023.0.6                amazonlinux     41 k
Installing dependencies:
ruby3.2-default-gems                   noarch          3.2.0-184.amzn2023.0.6                amazonlinux     33 k
ruby3.2-libs                           x86_64          3.2.0-184.amzn2023.0.6                amazonlinux     3.9 M
ruby3.2-rubygem-io-console              x86_64          0.6.0-184.amzn2023.0.6                amazonlinux     24 k
ruby3.2-rubygem-json                    x86_64          2.6.3-184.amzn2023.0.6                amazonlinux     52 k
ruby3.2-rubygem-psych                   x86_64          5.0.1-184.amzn2023.0.6                amazonlinux     50 k
Installing weak dependencies:
ruby3.2-rubygem-bigdecimal              x86_64          3.1.3-184.amzn2023.0.6                amazonlinux     67 k
ruby3.2-rubygem-bundler                 noarch          2.4.19-184.amzn2023.0.6                amazonlinux     384 k
ruby3.2-rubygem-rdoc                     noarch          6.5.1.1-184.amzn2023.0.6              amazonlinux     459 k
ruby3.2-rubygems                         noarch          3.4.19-184.amzn2023.0.6                amazonlinux     259 k
ruby4.0-default-gems                    noarch          4.0.1-32.amzn2023.0.3                 amazonlinux     28 k
=====
Transaction Summary
=====
Install 11 Packages

Total download size: 5.3 M
Installed size: 18 M
Downloading Packages:
(1/11): ruby3.2-3.2.0-184.amzn2023.0.6.x86_64.rpm                1.0 MB/s | 41 kB  00:00
(2/11): ruby3.2-default-gems-3.2.0-184.amzn2023.0.6.noarch.rpm   761 kB/s | 33 kB  00:00
(3/11): ruby3.2-libs-3.2.0-184.amzn2023.0.6.x86_64.rpm          40 MB/s | 3.9 MB  00:00

```

```
[ec2-user@ip-172-31-82-29 ~]$ cd /home/ec2-user
[ec2-user@ip-172-31-82-29 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-82-29 ~]$ REGION=us-east-1
[ec2-user@ip-172-31-82-29 ~]$ wget https://aws-codedeploy-$REGION.s3.$REGION.amazonaws.com/latest/install
--2026-09-10 22:18:43-- https://aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com/latest/install
Resolving aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com (aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com)... 52.217.206.202, 54.231.136.42, 52.217.225.26, ...
Connecting to aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com (aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com)|52.217.206.202|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 19045 (19K) []
Saving to: 'install'

install                                100%[=====] 18.60K --KB/s in 0.001s

2026-09-10 22:18:43 (31.7 MB/s) - 'install' saved [19045/19045]

[ec2-user@ip-172-31-82-29 ~]$ head -1 install
#!/usr/bin/env ruby
[ec2-user@ip-172-31-82-29 ~]$ chmod +x ./install
[ec2-user@ip-172-31-82-29 ~]$ sudo ./install auto
I, [2026-09-10T22:20:00.971637 #34168] INFO -- : Starting Ruby version check.
W, [2026-09-10T22:20:00.971732 #34168] WARN -- : The Ruby version in /usr/bin/ruby3.2 is 3.2.8, . Attempting to install anyway.
I, [2026-09-10T22:20:00.971773 #34168] INFO -- : Starting update check.
I, [2026-09-10T22:20:00.971803 #34168] INFO -- : Attempting to automatically detect supported package manager type for system...
I, [2026-09-10T22:20:00.987020 #34168] INFO -- : Checking AWS_REGION environment variable for region information...
I, [2026-09-10T22:20:00.987071 #34168] INFO -- : Checking EC2 metadata service for region information...
```

```
[ec2-user@ip-172-31-82-29 ~]$ sudo systemctl status codedeploy-agent
● codedeploy-agent.service - AWS CodeDeploy Host Agent
   Loaded: loaded (/usr/lib/systemd/system/codedeploy-agent.service; enabled; preset: disabled)
   Active: active (running) since Thu 2026-09-10 22:20:03 UTC; 2min 33s ago
     Main PID: 34308 (ruby)
       Tasks: 3 (limit: 1059)
      Memory: 67.1M
         CPU: 1.258s
    CGroup: /system.slice/codedeploy-agent.service
            └─34308 "codedeploy-agent: master 34308"
               34310 "codedeploy-agent: InstanceAgent::Plugins::CodeDeployPlugin::CommandPoller of master 34308"

Sep 10 22:20:03 ip-172-31-82-29.ec2.internal systemd[1]: Starting codedeploy-agent.service - AWS CodeDeploy Host Agent...
Sep 10 22:20:03 ip-172-31-82-29.ec2.internal systemd[1]: Started codedeploy-agent.service - AWS CodeDeploy Host Agent.
```

Pueda hablar y descargar del nuevo bucket S3, que tendrá almacenado el jar de la app.

Al final, al paso que llegamos fue la preparación del repo con la cual conectaríamos un repositorio de Github con unos ajustes a los archivos de buildspec, appsec y taskflow.service junto con una carpeta de scripts para la simulación de hooks para la app (No se pudo realizar la conexión porque CodeDeploy ya no está disponible para los planes gratis):

The screenshot shows a GitHub repository interface for 'taskflow-aws-AndresIng95'. The repository is public and has 6 commits. The file structure is as follows:

File/Folder	Description	Last Commit
.settings	TaskFlow API, punto de partida	yesterday
postman	TaskFlow API, punto de partida	yesterday
scripts	ci: buildspec, appspec, hooks y taskflow.service	5 hours ago
src	TaskFlow API, punto de partida	yesterday
.classpath	TaskFlow API, punto de partida	yesterday
.dockerignore	TaskFlow API, punto de partida	yesterday
.env.example	TaskFlow API, punto de partida	yesterday
.gitattributes	TaskFlow API, punto de partida	yesterday
.gitignore	ci: buildspec, appspec, hooks y taskflow.service	5 hours ago
.project	TaskFlow API, punto de partida	yesterday

On the right side, the 'About' section is empty, and the 'Releases' and 'Packages' sections show 'No releases published' and 'No packages published' respectively. The 'Contributors' section shows 1 contributor.

De aquí los siguientes pasos sería la creación de un rol para CodeDeploy, junto con la creación de un deployment group en la nube, un CodePipeline con su CodeBuilder integrado, y la conexión a Github con AWS, dando ya los pasos que vimos en la imagen unas paginas atrás.

De manera simple, estos pasos serian de manera simple tenemos un código en maquina que le hacemos git push, donde se suben a github, se activa CodePipeline, adentro Codebuild lee estos cambios con buildspec.yml, empaqueta con mvn y crea una zip para subirla al bucket S3, CodeDeploy baja esta zip del S3 y la manda a la instancia de EC2 que tenemos, donde adentro hay un agente que instalamos.

Este agente lee appspec.yml, donde luego corre nuestros scripts que incluimos, para poder parar la aplicación, darle los permios que necesita, iniciar la aplicación, y la verificación de esta misma, en ese orden. (Esta la parte de parar la app primero para poder hacer una actualización a esta misma, si estás en tu segundo despliegue o más, tu vieja versión sigue viva, por la cual la detienes, y creas una nueva). Finalizando, tenemos una aplicación arriba que se actualiza cada vez que haces push a Github, pudiendo tener una aplicación con arquitectura en la nube.